



MODEL
CONTEXT
PROTOCOL



LEARN



BUILD



INTEGRATE

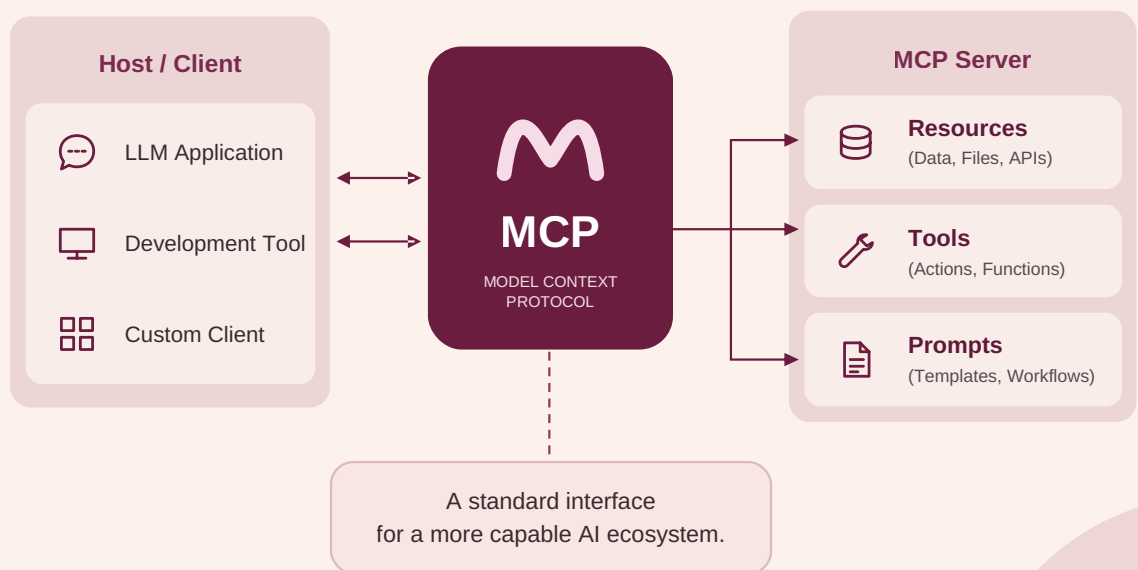


EXTEND

CHAPTER 08

Building Custom MCP Clients

Connect your own chatbot to local and remote MCP servers using LangChain's MCPAdapter and a Streamlit UI.



Premium Revision Notes

CONCEPTS

Clear Explanations

IMPLEMENTATION

Practical Code

DIAGRAMS

Visual Learning

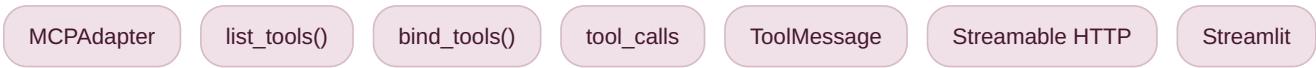
INTERVIEW PREP

Key Questions

Building Custom MCP Clients

Until now, every MCP server in this series was consumed through **Claude Desktop**, a finished host with an MCP client built in. That is convenient, but it is not what you ship when you build your own product. A custom chatbot, an internal assistant, or a backend agent has to contain its own client code: it must read a list of servers, open connections, discover tools, hand them to a model, execute the calls the model requests, and feed the results back.

This chapter builds exactly that, step by step. You will write a terminal client first (`client1.py`) to understand every moving part, then wrap the same logic in a **Streamlit** chat interface (`client2.py`). The client talks to three servers at once: a *local* math server over STDIO, a *remote* Expense Tracker over Streamable HTTP (exposed with a Cloudflare tunnel), and a third-party Manim animation server. Every implementation detail comes from the attached source files, which use the current **MCP Python SDK v2** for servers and LangChain’s first-party `langchain.mcp` adapter for the client.



Chapter	Position in the series	What it covers
5	The How · Part 1	Connecting an existing host (Claude Desktop) to existing servers
6	The How · Part 2	Building local MCP servers with MCP SDK v2
7	The How · Part 3	Building remote MCP servers and exposing them over HTTP
8	The How · Part 4	Building your own MCP client (this chapter)

1. From Ready-Made Host to Your Own Client

Why build a client at all?

In the MCP architecture a **host** is the application the user interacts with, and it owns the LLM. Inside the host, a **client** component maintains the connection to MCP servers and speaks the protocol. Claude Desktop bundles both. When you build your own chatbot, *you* become the host, so you must supply the client yourself and decide how the LLM, the UI and the servers are wired together.

The system we will build

The target is a chatbot with a normal conversational mode and a tool mode. Ordinary questions are answered by the model directly; questions that need arithmetic, expense tracking or animation cause the model to request a tool, which the client forwards to the correct MCP server. One extra server can be added by editing a single dictionary, with no change to the control flow.

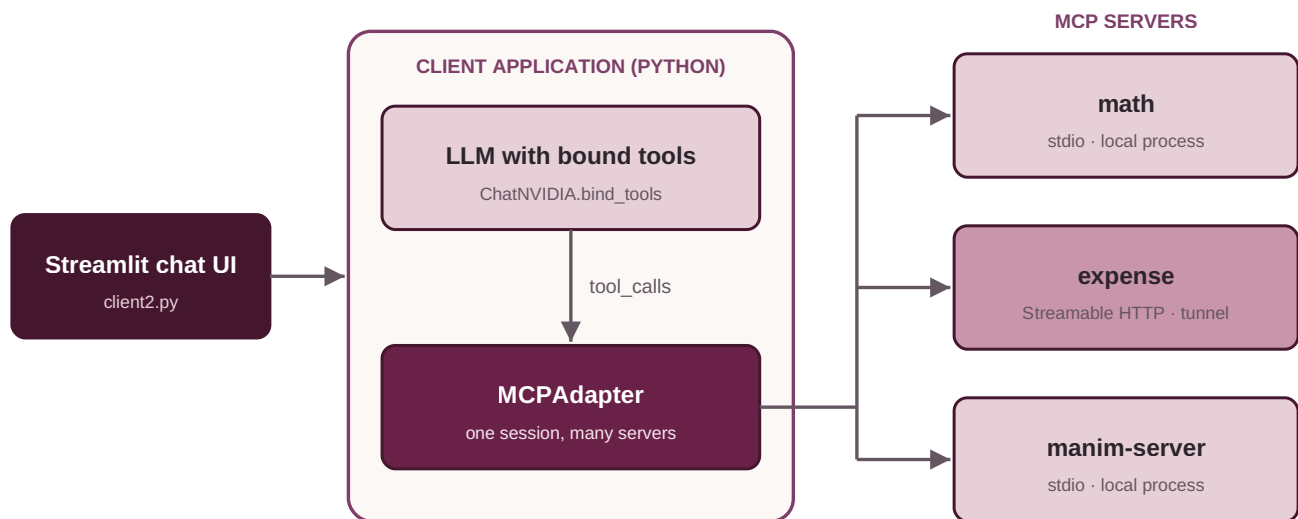


Figure 8.1 — Target system: a Streamlit UI drives a Python client that reaches three MCP servers through one MCPAdapter.

IMPORTANT

A custom client executes *whatever tool the model asks for*. There is no built-in approval dialog as in Claude Desktop. Trust decisions (which servers to register, which tools may run unattended) become your responsibility.

What changes when you leave Claude Desktop

Concern	Claude Desktop	Your custom client
Who owns the LLM?	Anthropic's app (Claude models)	You choose any model that supports tool calling
Server registration	<code>claude_desktop_config.json</code> or connectors	A Python dict with the same <code>mcpServers</code> shape
Tool-calling loop	Built in and invisible	You write it: call model, run tools, return results
Permission prompts	Shown to the user before tool use	None by default; you must design them
User interface	Fixed desktop chat window	Anything: terminal, Streamlit, web app, API

🔄 Quick Revision

- A host owns the LLM and UI; the client inside it speaks MCP to servers.
- Building your own chatbot means writing the client code and the tool-calling loop yourself.
- The server registry keeps the same `mcpServers` JSON shape used by Claude Desktop.
- Permission prompts and safety checks are not automatic in a custom client.

2. Choosing the Client Library and Setting Up

Three ways to write an MCP client in Python

Option	Level	Choose it when
Official MCP Python SDK client	Low-level sessions and transports	You need full control of the protocol
FastMCP <code>Client</code>	Pythonic client with transports, auth, caching	You want a direct client without an LLM framework
LangChain <code>MCPAdapter</code>	Returns ready-to-bind LangChain tools	You already use LangChain or LangGraph (used here)

This chapter uses `langchain.mcp.MCPAdapter`. It ships inside the main LangChain package (installed through the `mcp` extra), is built on FastMCP, and accepts the same `mcpServers` configuration format as Claude Desktop. It turns every MCP tool into a standard LangChain tool, which can then be passed to `bind_tools()` on any tool-calling chat model. Because the protocol concepts are the same everywhere, the loop you learn here transfers directly to the other two options.

COMMON MISTAKE

Many older tutorials import MCP helpers from the standalone `langchain_mcp_adapters` package. MCP support now lives inside LangChain under `langchain.mcp`, and the examples in this chapter use only that namespace. If an import does not resolve, check that you installed `langchain[mcp]`.

Project setup (Windows)

```
pip install uv
mkdir mcp-client && cd mcp-client
uv init
uv add "langchain[mcp]" langchain-nvidia-ai-endpoints streamlit python-dotenv
where uv
```

`uv init` creates the project metadata, and `uv add` installs the dependencies into the project environment. `where uv` prints the full path to `uv.exe`, which you will need later because the STDIO server entry launches the math server through `uv`. Create a `.env` file next to your scripts; `load_dotenv()` reads it at start-up so the key never appears in source code.

```
# .env (never commit this file)
NVIDIA_API_KEY=<YOUR_API_KEY>
```

BEST PRACTICE

Add `.env` to `.gitignore` immediately. Any chat model that supports tool calling can replace `ChatNVIDIA`; only the import and the constructor line change, while the MCP half of the code stays identical.

IMPORTANT

Everything in this chapter is **asynchronous**. MCP connections, tool calls and model calls are all awaited (`await`), and the entry point is started with `asyncio.run(main())`. If `async/await` is unfamiliar, review it first; the rest of the code is easier to follow once you do.

🔄 Quick Revision

- `MCPAdapter` is LangChain's first-party MCP client (from `langchain.mcp import MCPAdapter`), built on FastMCP.
- Install with `uv add "langchain[mcp]"` plus the chat-model package, Streamlit and `python-dotenv`.
- Secrets go in `.env` and are loaded with `load_dotenv()`; never hard-code keys.
- All MCP and model calls are `async`; run the entry point with `asyncio.run`.

3. The Servers the Client Will Use

3.1 Local math server (STDIO)

The first server is a small arithmetic server written with MCP SDK v2. `MCPServer("arith")` creates the server object, and each function decorated with `@mcp.tool()` becomes a tool whose name, parameters and description are derived from the function name, type hints and docstring. The helper `_as_number()` accepts ints, floats or numeric strings, which makes the tools tolerant of model-generated arguments that arrive as text. Calling `mcp.run()` with no arguments serves over STDIO, so the client launches the file as a subprocess.

Tool	Computes	Notes
add / subtract / multiply	$a + b$, $a - b$, $a \times b$	All arguments pass through <code>_as_number</code>
divide	a / b	Raises <code>ValueError</code> when <code>b</code> is zero
power	$a ** b$	Exponentiation
modulus	$a \% b$	Remainder; <code>21 % 7</code> returns 0

Test the server on its own with MCP Inspector (Chapter 6) before involving a client. If a tool misbehaves in Inspector, no amount of client debugging will fix it, so isolating the server first saves time.

```

from __future__ import annotations
from mcp.server import MCPServer

mcp = MCPServer("arith")

def _as_number(x):
    """Convert an int, float, or numeric string into a float."""
    if isinstance(x, (int, float)):
        return float(x)

    if isinstance(x, str):
        try:
            return float(x.strip())
        except ValueError:
            raise ValueError(f"Invalid numeric value: {x}")

    raise TypeError("Expected a number (int/float or numeric string)")

@mcp.tool()
async def add(a: float, b: float) -> float:
    """Return a + b."""
    return _as_number(a) + _as_number(b)

# ... subtract, multiply, power and modulus follow the same pattern ...

@mcp.tool()
async def divide(a: float, b: float) -> float:
    """Return a / b."""
    b = _as_number(b)

    if b == 0:
        raise ValueError("Cannot divide by zero")

    return _as_number(a) / b

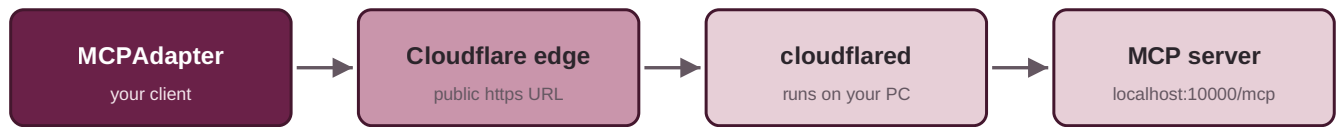
if __name__ == "__main__":
    mcp.run()

```

3.2 Remote Expense Tracker (Streamable HTTP through a tunnel)

The second server is the Expense Tracker built in Chapter 7. It runs on your own machine with `run(transport="streamable-http", host="0.0.0.0", port=10000)`, and its MCP endpoint is the `/mcp` path. Instead of deploying it to a hosting platform, it is made reachable with a **Cloudflare quick tunnel**:

```
cloudflared tunnel --url http://localhost:10000
# prints a public address such as https://<random-words>.trycloudflare.com
# the MCP endpoint for the client is that address + /mcp
```



cloudflared opens an outbound connection to Cloudflare, so no inbound port is exposed on your network

Figure 8.2 — A quick tunnel forwards public HTTPS requests to the local MCP server through an outbound cloudflared connection.

COMMON MISTAKE

A quick-tunnel address is **temporary**: it changes every time `cloudflared` restarts. When the `expense` server suddenly fails to connect, the first thing to check is whether the `url` in `SERVERS` still matches the address printed by the current tunnel.

IMPORTANT

A quick tunnel makes the server reachable by *anyone who knows the URL*, and the Chapter 7 server has no authentication. Use tunnels for learning and demos only, never for a server that holds real data.

3.3 Third-party Manim server (STDIO)

The third server is not written by you: a community Manim server that renders animations. It is registered exactly as it is in `claude_desktop_config.json`, with a `command`, `args`, and an `env` block that tells it where the Manim executable lives. The point is portability: any server you already configured for Claude Desktop can be attached to your own client by copying its entry.

3.4 Hosting options beyond a tunnel

A tunnel keeps a server on your laptop. When you want an always-on URL, the main alternatives are below. Free tiers and prices change often, so verify current terms before relying on them; the platform-specific deployment walkthroughs are intentionally left out.

Option	Cost model	Best for	Watch out for
Cloudflare quick tunnel	Free	Demos and local testing	Random URL that changes; no uptime
Cloudflare Workers	Free daily request allowance; paid plans above it	TypeScript servers at the edge	Python servers need containers, which are a paid feature
FastMCP Cloud	Free personal tier	Python FastMCP servers, one-command deploy	Tied to its ecosystem

Render	Free hobby tier; paid for always-on	Simple GitHub-to-URL deploys	Free services sleep when idle
Railway	Small trial credit, then usage-based	Python/Docker servers with persistence	Free credit is small
Fly.io	Pay-as-you-go, no free tier for new users	Regional or per-user isolation	Needs a card and more configuration
Cloud Run / VPS	Pay per use or fixed monthly	Enterprise or fully controlled hosting	You manage security and scaling

BEST PRACTICE

Whatever the hosting choice, a public MCP endpoint needs authentication (for example OAuth or a bearer token) before it carries real data. That topic builds on Chapter 7’s no-auth discussion and is an advanced follow-up.

🔄 Quick Revision

- MCP SDK v2 servers use `MCPServer` with `@mcp.tool()`; `mcp.run()` defaults to STDIO.
- A STDIO server is started by the client as a subprocess; a remote server must already be running.
- A Cloudflare quick tunnel gives a free, temporary public URL; the MCP endpoint is that URL plus `/mcp`.
- Third-party servers attach by copying their `command/args/env` entry.
- Tunnels suit demos; permanent hosting needs a platform and authentication.

4. Telling the Client About the Servers

The client needs a registry of servers. It is a plain Python dictionary with a top-level `mcpServers` key, the same schema as Claude Desktop’s config file. The adapter reads it and decides how to reach each entry.

```

import json
import asyncio
from dotenv import load_dotenv

from langchain.mcp import MCPAdapter
from langchain_core.messages import ToolMessage
from langchain_nvidia_ai_endpoints import ChatNVIDIA

load_dotenv()

SERVERS = {
    "mcpServers": {
        "math": {
            "command": "C:/Users/saura/.local/bin/uv.exe",
            "args": [
                "run",
                "mcp",
                "run",
                "C:/Users/saura/OneDrive/Desktop/AI Engineering/MCP/3.MCP Clients/Local
Server/main.py",
            ],
        },
        "expense": {
            "transport": "streamable-http",
            # We ran the MCP server through a Cloudflare tunnel rather than
            # actually deploying it and using a live web URL, so this URL
            # changes everytime you do that.
            "url": "https://<your-tunnel-subdomain>.trycloudflare.com/mcp"
        },
        "manim-server": {
            "command": "C:/Users/saura/AppData/Local/Programs/Python/Python311/python.exe",
            "args": [
                "C:/Users/saura/Desktop/manim-mcp-server/src/manim_server.py"
            ],
            "env": {
                "MANIM_EXECUTABLE": "C:/Users/saura/AppData/Local/Programs/Python/Python311/
Scripts/manim.exe"
            }
        }
    }
}

```

Field	Meaning	Used by
Entry key (<code>math</code> , <code>expense</code> , ...)	A unique server name; it also becomes the tool-name prefix (see below)	All entries
<code>command</code> + <code>args</code>	Executable and arguments the client runs to start the server over STDIO	Local servers
<code>env</code>	Environment variables passed to that subprocess	Local servers needing settings
<code>transport</code> + <code>url</code>	Connect to an already-running server over Streamable HTTP	Remote servers

IMPORTANT

Tool names are prefixed with the server key. When several servers are registered, FastMCP namespaces every tool as `{server}_{tool}`, so the `math` server's `add` appears as `math_add` and two servers can safely expose tools with the same name. The client code never hard-codes these names; it reads them from `tool.name`, and printing `named_tools.keys()` shows exactly what the model will see.

How local and remote entries differ at run time

For a `command` entry, the client is the parent process: it starts the server, owns its lifetime and stops it at the end. For a `url` entry, the server runs independently and the client only opens an HTTP session to it. This is the same distinction between local and remote servers established in Chapters 3 and 7, now visible as two different shapes of dictionary entry.

BEST PRACTICE

Write Windows paths with forward slashes (`C:/Users/...`) inside Python strings. Backslashes need escaping in Python and in JSON, and a single missed escape produces a path that silently fails to launch.

🔄 Quick Revision

- The registry is a dict with `mcpServers`; each entry is either `command/args/env` (STDIO) or `transport/url` (Streamable HTTP).
- The entry key doubles as the tool prefix: `math` + `add` becomes `math_add`.
- The client owns local server processes but only connects to remote ones.
- Use forward slashes in Windows paths.

5. Client v1, Step 1: Connect, Discover Tools, Bind to the LLM

The first half of `main()` establishes everything the model will need. Read the code below top to bottom: it opens the connections, lists tools, builds a lookup table, creates the model and attaches the tools.

```

async def main() -> None:
    async with MCPAdapter(SERVERS) as adapter:

        # Discover MCP tools and convert them to LangChain tools
        tools = await adapter.list_tools()

        llm = ChatNVIDIA(
            model="nvidia/nemotron-3.5-lightning-30b-a3b"
        )

        llm_with_tools = llm.bind_tools(tools)

        named_tools = {}

        for tool in tools:
            named_tools[tool.name] = tool

        print("\n Available tools : ", named_tools.keys())

        # prompt = "What is the product of numbers 238 and 493?"
        #         Use a tool from the available tools."
        # prompt = "Add an expense for rupees 800 for groceries
        #         on 3rd september."

        prompt = "Draw a triangle rotating in place using the manim tool."

        response = await llm_with_tools.ainvoke(prompt)

```

What each stage does

- **async with MCPAdapter(SERVERS) as adapter** opens the session. Entering the block starts the STDIO subprocesses, connects to the remote URL and performs the protocol handshake (the initialization phase from Chapter 4). Leaving the block shuts everything down, so servers never leak as orphan processes.
- **adapter.list_tools()** performs tool discovery across all servers and returns LangChain tool objects, each carrying a name, description and argument schema.
- **named_tools** is a dictionary from tool name to tool object. The model will later answer with only a *name* and arguments, so the client needs fast lookup from name to the callable object.
- **llm.bind_tools(tools)** returns a new model object that advertises those tool schemas on every request. It takes the *list* of tools, not the dictionary.
- **ainvoke(prompt)** sends the prompt to the model and returns an **AIMessage**.

IMPORTANT

The model **never runs a tool**. It can only *ask* for one by returning a **tool_calls** attribute. Your code is the executor. Whether it asks at all is the model’s own decision: for a trivial sum it may simply answer from its own knowledge, in which case **tool_calls** is empty. Naming the capability in the prompt, such as “use a tool from the available tools”, nudges it to use the server.

🔄 Quick Revision

- Entering `async` with `MCPAdapter(...)` runs initialization; leaving it shuts the servers down.
- `list_tools()` returns LangChain tools from every registered server in one flat list.
- Keep a `name → tool` dict to execute the tool the model names.
- `bind_tools` takes the list of tools and returns a new tool-aware model.
- The model requests tools; the client executes them.

6. Client v1, Step 2: The Tool-Calling Loop

A single tool-using answer needs **two** model calls with tool execution in between. The first call yields a tool request; the second call, made after you attach the result, yields the final natural-language answer.

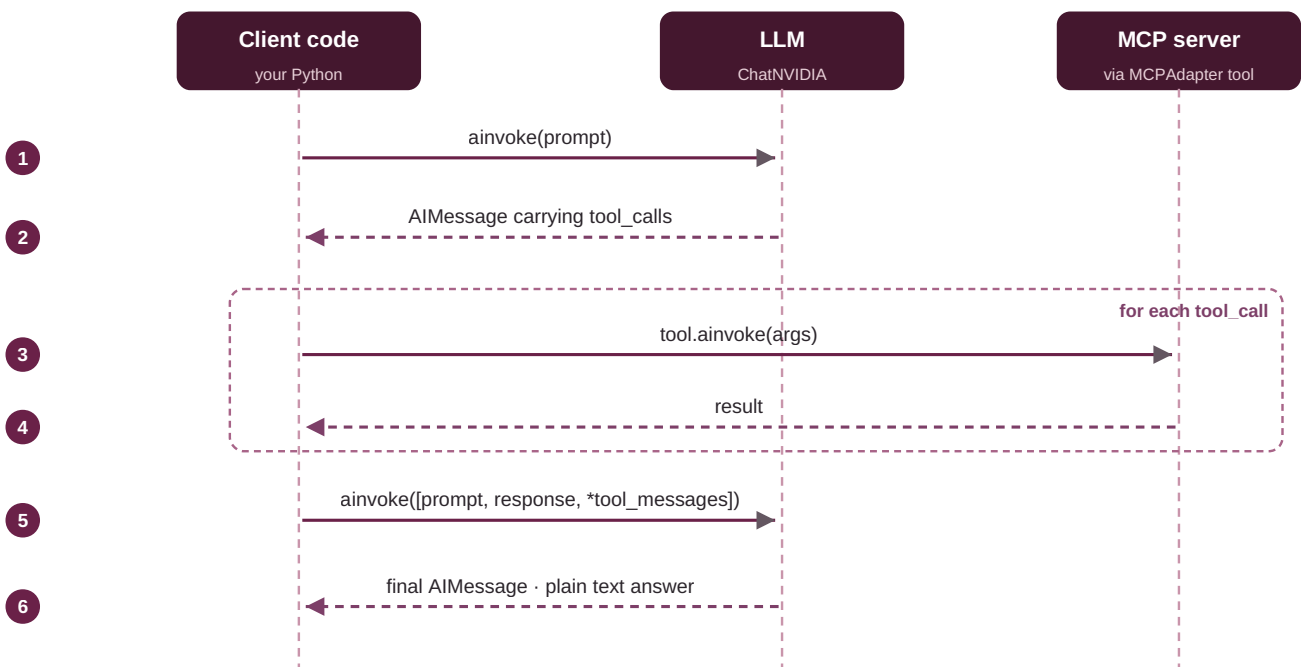


Figure 8.3 — One tool-calling round: the model requests a tool, the client executes it on the MCP server, and the model phrases the result.

Before reading the code, fix the ordering rule in mind. The conversation sent back to the model must tell a coherent story, and a tool result is only meaningful next to the request that produced it.

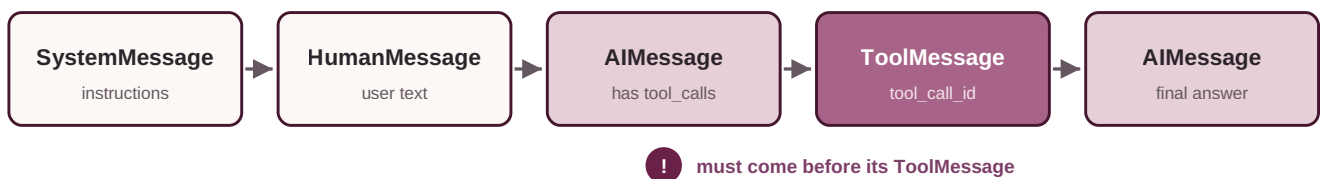


Figure 8.4 — Required order of the conversation when tools are used: the tool-requesting AIMessage precedes its ToolMessage.

```

if not getattr(response, "tool_calls", None):
    print(f"\n LLM reply : {response.content}")
    return

tool_messages = []

for tc in response.tool_calls:
    selected_tool = tc["name"]
    selected_tool_args = tc.get("args") or {}
    selected_tool_id = tc["id"]

    result = await named_tools[selected_tool].ainvoke(selected_tool_args)

    tool_messages.append(ToolMessage(content = json.dumps(result), tool_call_id =
selected_tool_id))

final_response = await llm_with_tools.ainvoke([prompt, response, *tool_messages])
print(f"Final response : {final_response.content}")

if __name__ == "__main__":
    asyncio.run(main())

```

Walking through the code

1. **No tool requested.** `getattr(response, "tool_calls", None)` is empty or missing for ordinary questions such as “What is the capital of India?”. The code prints the reply and returns early. Without this guard the later lines would fail on a plain answer.
2. **Read each request.** `response.tool_calls` is a *list* because a model may ask for several tools in one turn. Every item is a dictionary holding `name`, `args` and a unique `id`.
3. **Execute.** `named_tools[name].ainvoke(args)` sends the call to the owning MCP server and awaits the result.
4. **Wrap the result.** `ToolMessage(content=..., tool_call_id=...)` labels the result with the `id` of the request it answers.
5. **Second model call.** The list `[prompt, response, *tool_messages]` replays the whole story: the original question, the model’s tool request, and the tool outputs. A bare string in that list is treated as a user message.

COMMON MISTAKE

Omitting `tool_call_id`, or giving the tool’s *name* instead of the request’s *id*, makes the second call fail. The `id` is how the model matches each result to the request it made, which matters when several tools were requested at once.

Prompt in <code>client1.py</code>	Expected behaviour
“What is the product of numbers 238 and 493? Use a tool from the available tools.”	Model requests the math server’s multiply tool; final answer is 117334
“Add an expense for rupees 800 for groceries on 3rd september.”	Model requests an Expense Tracker tool (remote server)
“Draw a triangle rotating in place using the manim tool.”	Model requests the Manim server’s tool; an animation file is produced by that server
“What is the capital of India?”	No <code>tool_calls</code> ; early return with the model’s own answer

BEST PRACTICE

Serialize results with `json.dumps(result, default=str)` as `client2.py` does. Tool outputs can contain values the JSON encoder cannot handle on its own, and `default=str` converts them instead of raising an exception.

INTERVIEW TIP

“Who executes a tool call in an MCP application?” The host application’s client code, not the LLM. The model only emits a structured request; the client validates it, invokes the server, and returns the output as a tool message.

🔄 Quick Revision

- Tool use takes two model calls: request, then final answer after results are attached.
- `tool_calls` is a list of dicts with `name`, `args`, `id`.
- Each result is a `ToolMessage` whose `tool_call_id` equals the request `id`.
- The second call must replay prompt, tool-requesting `AIMessage`, then `ToolMessages`, in that order.
- Guard with an early return when `tool_calls` is empty.

7. Scaling Up: Many Servers and Many Tool Calls

One adapter, one flat tool list

Nothing in the control flow of Section 6 depends on the number of servers. `MCPAdapter(SERVERS)` merges all registered servers behind one aggregate connection, so `list_tools()` already returns math, expense and Manim tools together, and each tool object knows which server it belongs to. Adding a server is therefore a configuration change, not a code change.

Several tool calls in one response

Because `tool_calls` is a list and the code iterates over it, the same loop handles one request or five. A prompt such as “add 2 and 3, then add an expense” may yield calls to two different servers in the same response; the loop executes them

in order and appends one `ToolMessage` per call. Executing them one after another keeps the code simple; independent calls could be run concurrently with `asyncio.gather` if latency matters.

Checklist: attach another MCP server

1. Copy the server's `command/args/env` (or `url`) entry from its documentation or from your `claude_desktop_config.json`.
2. Paste it into `SERVERS["mcpServers"]` under a unique, descriptive key.
3. Make sure every path and executable in the entry exists on the machine running the client.
4. Run the client and print `named_tools.keys()` to confirm the new prefixed tools appear.
5. Prompt using the capability by name so the model chooses the new tool.

REAL WORLD INSIGHT

Every server you attach adds its tool schemas to every model request. A long tool list consumes context and can make tool selection less reliable, so register only the servers an application actually needs.

🔄 Quick Revision

- Adding a server only requires a new entry in `SERVERS`.
- The loop over `tool_calls` supports multiple tools per response, even across servers.
- Confirm attachment by printing the prefixed tool names.
- More tools mean more context per request.

8. Client v2: A Streamlit Chat Interface

The terminal client proves the mechanism but handles one question per run. `client2.py` keeps the identical MCP and LLM logic, moves it into one reusable coroutine, and adds a chat front-end with memory. Its structure follows a single idea: **the conversation is a list of LangChain message objects** that is stored by Streamlit, passed to the model on every turn, and rendered back to the screen.

Message type	Created by	Shown in the chat UI?
SystemMessage	App start-up (SYSTEM_PROMPT)	No
HumanMessage	The user's chat input	Yes
AIMessage with tool_calls	The model, as a tool request	No (intermediate)
ToolMessage	The client, wrapping a tool result	No
AIMessage (final)	The model's answer, stored by the client	Yes

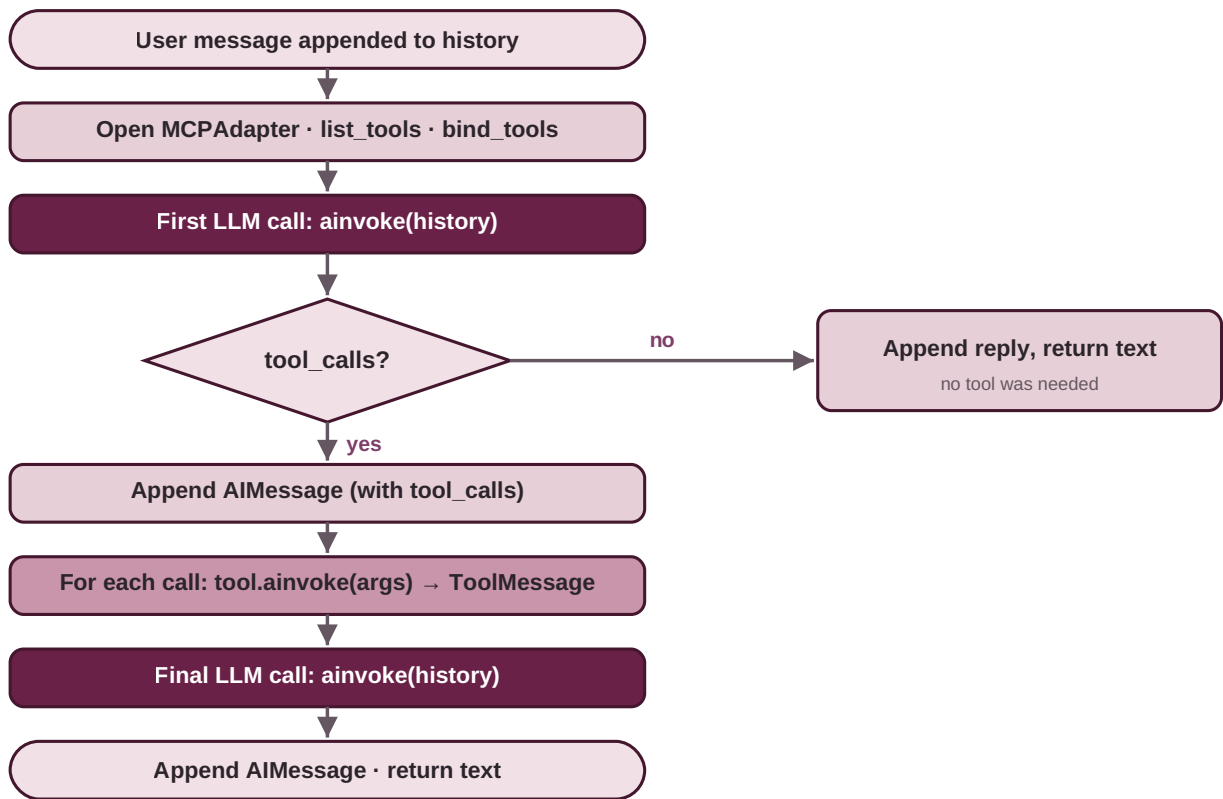


Figure 8.5 — Control flow of `process_message()`: one optional tool round per user message.

Imports, system prompt and the first model call

```

import json
import asyncio
import streamlit as st

from dotenv import load_dotenv
from langchain.mcp import MCPAdapter
from langchain_core.messages import (
    HumanMessage,
    AIMessage,
    ToolMessage,
    SystemMessage,
)
from langchain_nvidia_ai_endpoints import ChatNVIDIA

load_dotenv()

# SERVERS = { ... identical mcpServers configuration to client1.py ... }

SYSTEM_PROMPT = (
    "You have access to tools. "
    "When you choose to call a tool, do not narrate status updates. "
    "After tools run, return only a concise final answer."
)

```

```

async def process_message(history):

    # MCPAdapter manages the MCP connections
    async with MCPAdapter(SERVERS) as adapter:

        # 1. Get MCP tools and convert them to LangChain tools
        tools = await adapter.list_tools()

        tool_by_name = {
            tool.name: tool
            for tool in tools
        }

        # 2. Create LLM
        llm = ChatNVIDIA(
            model="nvidia/nemotron-3.5-lightning-30b-a3b"
        )

        # Give MCP tools to the LLM
        llm_with_tools = llm.bind_tools(tools)

        # 3. First LLM call
        first_response = await llm_with_tools.ainvoke(history)

        tool_calls = getattr(first_response, "tool_calls", None)

        # 4. No tool required
        if not tool_calls:

            history.append(first_response)

            return first_response.content

```

The `SYSTEM_PROMPT` tells the model to stay quiet while calling tools and to answer concisely afterwards, so the user sees only a final answer. `process_message(history)` receives the whole conversation, so the model has context from earlier turns. When no tool is requested, the reply is appended to the history and returned.

Two details differ from `client1.py`. First, arguments are defensively parsed with `json.loads` if they arrive as a JSON string. Second, the intermediate `AIMessage` that carries `tool_calls` is added to `history` *before* the `ToolMessage` objects, which preserves the order shown in Figure 8.4. The final answer is stored as a plain `AIMessage` so later turns see a clean transcript.

Tool execution and the final answer

```
# 5. LLM requested one or more tools

# IMPORTANT:
# The assistant message containing tool_calls
# must come before ToolMessages.

history.append(first_response)

tool_messages = []

for tool_call in tool_calls:

    tool_name = tool_call["name"]
    tool_args = tool_call.get("args") or {}
    tool_call_id = tool_call["id"]

    # Sometimes arguments can arrive as a JSON string
    if isinstance(tool_args, str):
        try:
            tool_args = json.loads(tool_args)
        except json.JSONDecodeError:
            pass

    # Find the requested LangChain tool
    tool = tool_by_name[tool_name]

    # Execute MCP tool
    result = await tool.ainvoke(tool_args)

    # Send result back as ToolMessage
    tool_messages.append(
        ToolMessage(
            content=json.dumps(result, default=str),
            tool_call_id=tool_call_id,
        )
    )

# Add all tool results to conversation history
history.extend(tool_messages)

# 6. Final LLM call
final_response = await llm_with_tools.ainvoke(history)

history.append(
    AIMessage(
        content=final_response.content or ""
    )
)

return final_response.content or ""
```

The Streamlit layer

```
st.set_page_config(
    page_title="MCP Chat",
    page_icon="🤖",
    layout="centered",
)

st.title("🤖 MCP Chat")

# Initialize chat history
if "history" not in st.session_state:

    st.session_state.history = [
        SystemMessage(content=SYSTEM_PROMPT)
    ]

# Render previous chat
for message in st.session_state.history:

    if isinstance(message, SystemMessage):
        continue

    elif isinstance(message, HumanMessage):

        with st.chat_message("user"):
            st.markdown(message.content)

    elif isinstance(message, AIMessage):

        # Don't display intermediate AI tool-call messages
        if getattr(message, "tool_calls", None):
            continue

        with st.chat_message("assistant"):
            st.markdown(message.content or "")

# ToolMessages intentionally aren't rendered
```

```

user_text = st.chat_input("Type a message...")

if user_text:

    # Display user message immediately
    with st.chat_message("user"):
        st.markdown(user_text)

    # Add user message to history
    st.session_state.history.append(
        HumanMessage(content=user_text)
    )

    # Run MCP + LLM logic
    with st.chat_message("assistant"):

        with st.spinner("Thinking..."):

            final_answer = asyncio.run(
                process_message(st.session_state.history)
            )

            st.markdown(final_answer)

```

Streamlit re-runs the script from top to bottom on every interaction, so anything that must survive a rerun lives in `st.session_state`. The history loop skips the system prompt, skips AI messages that merely carry tool requests, and never draws `ToolMessage` objects, so the chat shows only human-readable turns. Because Streamlit is synchronous and the MCP code is asynchronous, `asyncio.run(...)` bridges the two for each message. Start the app with:

```
uv run streamlit run client2.py
```

Aspect	client1.py (terminal)	client2.py (Streamlit)
Interface	One hard-coded prompt per run	Interactive chat with memory
Conversation state	Local variables	<code>st.session_state.history</code> (message objects)
System prompt	None	<code>SYSTEM_PROMPT</code> as first message
Argument handling	Uses <code>args</code> as given	Parses JSON-string arguments
Result encoding	<code>json.dumps(result)</code>	<code>json.dumps(result, default=str)</code>
Tool servers	Opened once per run	Re-opened for every user message

KNOWN LIMITATIONS OF THIS VERSION

1. A new `MCPAdapter` session is created inside every `process_message()` call, so STDIO servers are launched again on each message; this is simple but adds latency. 2. There is exactly one tool round: if the final model call requests further tools, they are not executed. 3. Only the model's text is rendered, so files a tool produces (for example an animation) are not displayed in the UI. Each is a natural next improvement.

BEST PRACTICE

To remove limitation 2, replace the single follow-up call with a loop that repeats while the latest response contains `tool_calls`. LangChain also offers `create_agent`, which packages this loop, and `list_tools` supports client-side caching so tool discovery is not repeated on every run; consult the LangChain MCP documentation for the version you install.

🔄 Quick Revision

- The conversation is a list of `SystemMessage`, `HumanMessage`, `AIMessage` and `ToolMessage` objects kept in `st.session_state`.
- `process_message` opens the adapter, calls the model, optionally runs tools, calls the model again, and returns text.
- The UI hides system messages, tool-requesting AI messages and tool messages.
- `asyncio.run` bridges synchronous Streamlit and async MCP code.
- Limitations: adapter re-opened per message, one tool round, text-only rendering.

9. Troubleshooting and Hardening

Security checklist for a custom client

- Registering a STDIO server means running a command on your machine; only add servers you trust.
- Treat tool output as untrusted text that is fed back into the model; it could contain instructions.
- Put confirmation steps in front of tools that delete or modify data; LangChain's adapter can surface server questions as interrupts, which supports this pattern.
- Keep keys in environment variables and never print or log them.
- Authenticate any remote server before it holds real data.

Common failures and fixes

Symptom	Likely cause	Fix
Expense tools missing or connection error	Tunnel restarted, so the URL changed	Copy the new address into <code>SERVERS</code> and keep <code>/mcp</code> at the end
Local server fails to start	Wrong <code>uv.exe</code> path or script path	Use <code>where uv</code> ; use forward slashes; confirm the file exists
Model answers without calling a tool	Question is easy; tool use is optional	Name the tool or capability in the prompt; add a system prompt
Error when sending tool results back	Missing or wrong <code>tool_call_id</code>	Use the request's <code>id</code> exactly
Error on a plain question	Tool-loop code ran with no <code>tool_calls</code>	Add the early-return guard
Tool name not found in dict	Prefixed names assumed to be bare	Print <code>named_tools.keys()</code> and use <code>tool.name</code>
Authentication or key error from the model	Missing key in <code>.env</code>	Set <code>NVIDIA_API_KEY=<YOUR_API_KEY></code> and restart

🔄 Quick Revision

- Most failures trace to paths, a stale tunnel URL, or a missing `tool_call_id`.
- Print the discovered tool names whenever tools seem to be missing.
- Stdio servers execute local commands; trust matters.
- Add approval steps for destructive tools.

Key Takeaways

- **A custom chatbot needs its own MCP client.** `MCPAdapter` turns the servers in an `mcpServers` dictionary into LangChain tools.
- **Local entries use `command/args`; remote entries use `transport/url`.** The registry has the same shape as Claude Desktop's config.
- **The model asks, the client acts.** `tool_calls` carries name, args and id; the client runs the tool and replies with a `ToolMessage` bearing the same id.
- **Two model calls per tool round**, with history replayed in the order prompt, `AIMessage` with `tool_calls`, `ToolMessages`.
- **Tool names are prefixed by server key** in multi-server setups, which prevents collisions.

- **Streamlit keeps the message list in session state** and bridges to async code with `asyncio.run`.
- **Tunnels are for demos.** Real deployments need a host and authentication.

10. Revision Sheet

Six compact cards that summarize the chapter's mechanics.

Client skeleton

- `async` with `MCPAdapter(SERVERS)` as `adapter`
- `tools = await adapter.list_tools()`
- `llm.bind_tools(tools)`
- `asyncio.run(main())`

Server entry shapes

STDIO: `command`, `args`, optional `env`.

Remote: `transport: streamable-http`, `url`.

Tool request anatomy

`tool_calls` = list of `{name, args, id}`. Execute with `named_tools[name].ainvoke(args)`.

Return path

`ToolMessage(content=json.dumps(result, default=str), tool_call_id=id)`, then `ainvoke` the full history again.

Streamlit pattern

History in `st.session_state`; render Human and plain AI messages; run `asyncio.run(process_message(history))`; launch with `uv run streamlit run client2.py`.

Operational notes

Quick-tunnel URL changes on restart. Prefixed tool names. Forward slashes on Windows. Keys live in `.env`.

11. Interview Questions

Practice answering these aloud without looking at the notes.

- Q1.** In a custom chatbot built on MCP, which part is the host, which is the client, and which is the server?
- Q2.** Why does the LLM return `tool_calls` instead of running the tool, and who actually executes it?
- Q3.** Why must a `ToolMessage` include `tool_call_id`, and what breaks if it is missing?
- Q4.** How does a client start a STDIO server compared with reaching a remote Streamable HTTP server?
- Q5.** How are tool-name collisions avoided when two servers expose a tool called `add`?
- Q6.** What does the `async with MCPAdapter(...)` block manage, and which lifecycle stages does it correspond to?
- Q7.** What are the limitations of the one-round tool loop in `client2.py`, and how would you fix them?
- Q8.** What are the security risks of exposing a server through a public tunnel, and of registering arbitrary STDIO servers?
- Q9.** Why does the Streamlit app call `asyncio.run` for each user message, and what is the performance cost?

12. Further Reading

- LangChain MCP documentation: docs.langchain.com/oss/python/langchain/mcp
- Migration notes from the standalone adapter package: docs.langchain.com/oss/python/migrate/langchain-mcp-adapters
- Model Context Protocol specification and SDK documentation: modelcontextprotocol.io
- Streamlit documentation: docs.streamlit.io